Human Verification: `Trust`

The Certificate Lemma and Transport of Trust

Machine-generated from `lean/Trust.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.Trust` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. This is the central file of the trust-systems formalisation, providing the definitions and results referenced by all other modules.

Main results formalised here: (i) the certificate lemma (Lemma 2.2): τ is the greatest sound assignment; (ii) equivalence of the fixpoint form and the derivation-sup (max-min) form of τ; (iii) lax transport of trust along morphisms (Lemma 3.2); (iv) consolidation preservation (Theorem 3): a genuine position map preserves τ exactly; (v) the circular-citation counterexample; and (vi) the shape-only counterexample.

Contents

1 Overview **2**

2 The check-strength chain **3**
2.1 Class: `Chain` . . . 3

3 Meet and join on a chain **3**
3.1 Definitions: `cmin` and `cmax` . . . 3
3.2 Theorems: `cmin` and `cmax` properties . . . 3

4 Folds over lists **4**
4.1 Theorems: folds of `cmin` and `cmax` over lists . . . 4
4.2 Theorems: monotonicity lemmas for folds . . . 5

5 Trust systems **5**
5.1 Structure: `System` . . . 5
5.2 Definition: `System.Prem` . . . 6
5.3 Definition: `stepGrade` . . . 6

6 Canonical trust **7**
6.1 Definition: `tau` . . . 7
6.2 Theorem: `tau_eq` . . . 7

7 Soundness: the certificate lemma (Lemma 2.2) **8**
7.1 Definition: `Sound` . . . 8
7.2 Theorem: `sound_le_tau` (Lemma 2.2 ii) . . . 8
7.3 Theorem: `tau_sound` (Lemma 2.2 i) . . . 8

8 Derivations: max-min semantics agrees with fixpoint **9**
8.1 Inductive type: `Deriv` . . . 9
8.2 Definition: `foldPrems` . . . 9
8.3 Definition: `derivGrade` . . . 9
8.4 Theorem: `derivGrade_node` . . . 9
8.5 Theorems: `foldPrems_le` and `le_foldPrems` . . . 10
8.6 Theorem: `derivGrade_le_tau` . . . 10
8.7 Theorem: `tau_le_derivGrade` . . . 10

9 Morphisms and transport of trust **11**
9.1 Structure: `Mor` . . . 11
9.2 Theorem: `tau_transport` (Lemma 3.2) . . . 11

10 The circular-citation counterexample (Remark 2.3) **12**
10.1 Inductive type: `Two` . . . 12
10.2 Definition: `cyc` . . . 12
10.3 Theorem: `cyc_sound_top` . . . 13
10.4 Theorem: `cyc_no_deriv` . . . 13
10.5 Theorem: `cyc_not_wf` . . . 13

11 Consolidations **13**
11.1 Structure: `Consolidation` . . . 13
11.2 Definition: `Consolidation.toMor` . . . 14
11.3 Theorem: `Consolidation.tau_le` . . . 15
11.4 Theorem: `tau_consolidate` (Theorem 3) . . . 15

12 Composition of consolidations **15**
12.1 Definition: `Consolidation.id` . . . 15
12.2 Definition: `Consolidation.comp` . . . 16
12.3 Theorem: `tau_consolidate_comp` . . . 17

13 The shape-only counterexample **17**
13.1 Definitions: `ground` and `layer4` . . . 17
13.2 Theorems: `ground_wf` and `layer4_wf` . . . 18
13.3 Theorems: `ground_tau_hi` and `layer4_tau_bot` . . . 18

14 Translation conventions **18**

1 Overview

A *trust system* is a dependency structure graded in a poset of check strengths T. Claims have checking steps with premises; the grade records how strong a check each step received. The *canonical trust* $\tau(x)$ is the widest-bottleneck value: the best derivation's weakest link.

Two corrections to the prose notes discovered during formalisation:

1. Lemma 2.2 (i) needs T to be a **chain** (linear order), not merely a lattice: in a general lattice, a finite sup need not be attained by any element. All deployed check-posets are finite chains.
2. Lemma 3.2 needs $\phi(\bot) = \bot$ ("unchecked maps to unchecked"): monotonicity alone fails on underivable claims.

2 The check-strength chain

2.1 Class: `Chain`

Definition 2.1 (Chain)**.** A **chain** (bounded decidable linear order) is a type T with a relation $\leq$, a bottom element $\bot$, satisfying: reflexivity, transitivity, antisymmetry, totality, $\bot \leq a$ for all a, and decidable $\leq$.

Lean.

```
class Chain (T : Type) extends LE T where
  bot        : T
  le_refl    : ∀ a : T, a ≤ a
  le_trans   : ∀ {a b c : T}, a ≤ b → b ≤ c → a ≤ c
  le_antisymm : ∀ {a b : T}, a ≤ b → b ≤ a → a = b
  le_total   : ∀ a b : T, a ≤ b \/ b ≤ a
  bot_le     : ∀ a : T, bot ≤ a
  dec        : ∀ a b : T, Decidable (a ≤ b)
```

Haskell.

```
class (Eq t, Ord t, Bounded t) ⇒ Chain t where
  bot :: t
  bot = minBound
-- In Haskell, Ord + Bounded gives a total order with a bottom.
-- Decidability is automatic (all Haskell values are decidable).
```

Fidelity note. In Lean the chain structure is a typeclass with explicit proof fields. Lean's `instance` mechanism checks these proofs at class instantiation time. In Haskell, `Ord` provides totality and decidability; antisymmetry and transitivity are assumed (no proof terms).

3 Meet and join on a chain

3.1 Definitions: `cmin` and `cmax`

Definition 3.1 (Meet and join)**.** For $a, b : T$ in a chain:

$$\mathrm{cmin}(a,b) = \text{if } a \leq b \text{ then } a \text{ else } b \qquad \text{(meet, weaker of the two)},$$
$$\mathrm{cmax}(a,b) = \text{if } a \leq b \text{ then } b \text{ else } a \qquad \text{(join, stronger of the two)}.$$

Lean.

```
def cmin (a b : T) : T := if a ≤ b then a else b
def cmax (a b : T) : T := if a ≤ b then b else a
```

Haskell.

```
cmin :: (Ord t) ⇒ t → t → t
cmin a b = if a <= b then a else b

cmax :: (Ord t) ⇒ t → t → t
cmax a b = if a <= b then b else a

-- Equivalently: cmin = min, cmax = max.
```

3.2 Theorems: `cmin` and `cmax` properties

The following basic facts about `cmin` and `cmax` are proved and used throughout:

Theorem 3.2 (Meet and join properties)**.**

$\mathrm{cmin}(a,b) \le a$	(`cmin_le_left`)
$\mathrm{cmin}(a,b) \le b$	(`cmin_le_right`)
$c \le a \wedge c \le b \implies c \le \mathrm{cmin}(a,b)$	(`le_cmin`)
$a \le \mathrm{cmax}(a,b)$	(`le_cmax_left`)
$b \le \mathrm{cmax}(a,b)$	(`le_cmax_right`)
$\mathrm{cmax}(a,b) = a \vee \mathrm{cmax}(a,b) = b$	(`cmax_cases`)

The last property (`cmax_cases`) uses linearity of the chain and is the attainment fact that makes the soundness direction of the certificate lemma work.

Lean.

```
theorem cmin_le_left (a b : T) : cmin a b ≤ a
theorem cmin_le_right (a b : T) : cmin a b ≤ b
theorem le_cmin {a b c : T} (ha : c ≤ a) (hb : c ≤ b) : c ≤ cmin a
    b
theorem le_cmax_left (a b : T) : a ≤ cmax a b
theorem le_cmax_right (a b : T) : b ≤ cmax a b
theorem cmax_cases (a b : T) : cmax a b = a \/ cmax a b = b
```

Haskell.

```
-- Standard properties of min/max on a total order.
-- cmax_cases: max a b == a || max a b == b   (always True)
```

4 Folds over lists

4.1 Theorems: folds of `cmin` and `cmax` over lists

The following are helper lemmas used in the proofs of the main theorems. They express standard properties of iterated meets and joins over lists.

Theorem 4.1 (List fold lemmas)**.** Let l be a list and init an initial value.

$a \in l \implies a \le l.\mathrm{foldr}(\mathrm{cmax}, \mathrm{init})$	(`le_foldr_max`)
$l.\mathrm{foldr}(\mathrm{cmax}, \mathrm{init}) = \mathrm{init} \ \vee \ l.\mathrm{foldr}(\mathrm{cmax}, \mathrm{init}) \in l$	(`foldr_max_cases`)
$a \in l \implies l.\mathrm{foldr}(\mathrm{cmin}, \mathrm{init}) \le a$	(`foldr_min_le`)
$l.\mathrm{foldr}(\mathrm{cmin}, \mathrm{init}) \le \mathrm{init}$	(`foldr_min_le_init`)

`foldr_max_cases` is the key attainment lemma: on a chain, the max-fold is always attained by the initial value or a list member.

Lean.

```
theorem le_foldr_max {a : T} {l : List T} (h : a in l) (init : T) :
    a ≤ l.foldr cmax init

theorem foldr_max_cases : ∀ (l : List T) (init : T),
    l.foldr cmax init = init \/ l.foldr cmax init in l

theorem foldr_min_le {a : T} {l : List T} (h : a in l) (init : T) :
    l.foldr cmin init ≤ a

theorem foldr_min_le_init : ∀ (l : List T) (init : T),
    l.foldr cmin init ≤ init
```

Haskell.

```
-- le_foldr_max:     a `elem` l ⇒ a <= foldr cmax init l
-- foldr_max_cases: foldr cmax init l == init || foldr cmax init l `
   elem` l
-- foldr_min_le:     a `elem` l ⇒ foldr cmin init l <= a
-- foldr_min_le_init: foldr cmin init l <= init
```

4.2 Theorems: monotonicity lemmas for folds

Theorem 4.2 (Monotonicity of folds)**.** 1. **`foldr_min_map_mono`**: pointwise domination of mapped `cmin`-folds: if $f(a) \le g(a)$ for all $a \in l$ and $i_1 \le i_2$, then $(l.\mathrm{map}\, f).\mathrm{foldr}(\mathrm{cmin}, i_1) \le (l.\mathrm{map}\, g).\mathrm{foldr}(\mathrm{cmin}, i_2)$.

2. **`phi_foldr_min`**: a monotone map is lax for `cmin`-folds: $\phi(l.\mathrm{foldr}(\mathrm{cmin}, \mathrm{init})) \le (l.\mathrm{map}\, \phi).\mathrm{foldr}(\mathrm{cmin}, \phi(\mathrm{init}$ Holds for any monotone ϕ without meet-preservation.

3. **`mapCongr`**: congruence for `List.map` on members: if $f(a) = g(a)$ for all $a \in l$ then $l.\mathrm{map}\, f = l.\mathrm{map}\, g$.

Lean.

```
theorem foldr_min_map_mono {alpha : Type} {f g : alpha → T} {i1 i2 :
   T} :
    ∀ (l : List alpha),
    (∀ a, a in l → f a ≤ g a) → i1 ≤ i2 →
    (l.map f).foldr cmin i1 ≤ (l.map g).foldr cmin i2

theorem phi_foldr_min {T' : Type} [Chain T'] {phi : T → T'}
    (hmono : ∀ {a b : T}, a ≤ b → phi a ≤ phi b) :
    ∀ (l : List T) (init : T),
    phi (l.foldr cmin init) ≤ (l.map phi).foldr cmin (phi init)

theorem mapCongr {alpha beta : Type} {f g : alpha → beta} :
    ∀ (l : List alpha),
    (∀ a, a in l → f a = g a) → l.map f = l.map g
```

Haskell.

```
-- foldr_min_map_mono: pointwise f<=g, i1<=i2 ⇒
--   foldr cmin i1 (map f l) <= foldr cmin i2 (map g l)
-- phi_foldr_min: phi monotone ⇒
--   phi (foldr cmin init l) <= foldr cmin (phi init) (map phi l)
-- mapCongr: (forall a in l, f a == g a) ⇒ map f l == map g l
```

5 Trust systems

5.1 Structure: `System`

Definition 5.1 (Trust system)**.** A **trust system** over claims C graded in chain T consists of:

- a type `Step` of checking steps,
- concl : `Step` $\to C$ (the conclusion of each step),
- prems : `Step` $\to$ `List` C (finite premises),
- grade : `Step` $\to T$ (the check strength received),
- stepsInto : $C \to$ `List Step` (all steps concluding that claim — finite, as the registry is a finite artifact),

- coherence: $f \in \text{stepsInto}(x) \iff \text{concl}(f) = x$.

Lean.
```
structure System (C : Type) (T : Type) [Chain T] where
  Step        : Type
  concl       : Step → C
  prems       : Step → List C
  grade       : Step → T
  stepsInto   : C → List Step
  mem_stepsInto : ∀ {x : C} {f : Step},
    f in stepsInto x → concl f = x
  stepsInto_mem : ∀ f : Step, f in stepsInto (concl f)
```

Haskell.
```
data System c t = System
  { step         :: [()]          -- just the count; we index by ()
  , concl        :: () → c
  , prems        :: () → [c]
  , grade        :: () → t
  , stepsInto    :: c → [()]
  -- coherence conditions are Prop in Lean, omitted in Haskell
  }
-- In practice: Step is a concrete type like Bool or Evt.
```

Fidelity note. In Lean, `Step` is a field of the structure and can vary between instances. The Haskell approximation glosses over this type-level variation; in practice each instance uses a concrete `Step` type (e.g., `Bool` for the cyclic example, `Evt` for trajectories).

5.2 Definition: `System.Prem`

Definition 5.2 (Premise relation)**.** y is a **premise** of x in system S if there exists a checking step $f \in \text{stepsInto}(x)$ with $y \in \text{prems}(f)$:

$$S.\text{Prem}(y, x) \iff \exists f,\ f \in \text{stepsInto}(x) \wedge y \in \text{prems}(f).$$

Well-foundedness of this relation is the hypothesis of the certificate lemma. It is *load-bearing*: the circular counterexample (`cyc`) has $\neg\,\text{WellFounded}(\text{Prem})$.

Lean.
```
def System.Prem (y x : C) : Prop :=
  exists f, f in S.stepsInto x /\ y in S.prems f
```

Haskell.
```
prem :: System c t → c → c → Bool
prem s y x = any (\f → y `elem` prems s f) (stepsInto s x)
```

5.3 Definition: `stepGrade`

Definition 5.3 (Step grade under an assignment)**.** Given an assignment $t : C \to T$, the **grade** of step f under t is the meet of the step's grade with the assigned levels of all its premises:

$$\text{stepGrade}(t, f) = (\text{prems}(f).\text{map}(t)).\text{foldr}(\text{cmin},\ \text{grade}(f)).$$

This is the "weakest link" value: the step is only as strong as its weakest premise's assigned trust, bounded by the step's own check grade.

Lean.
```
def stepGrade (t : C → T) (f : S.Step) : T :=
  ((S.prems f).map t).foldr cmin (S.grade f)
```

Haskell.

```haskell
stepGrade :: System c t → (c → t) → StepOf s → t
stepGrade s t f = foldr cmin (grade s f) (map t (prems s f))
```

6 Canonical trust

6.1 Definition: `tau`

Definition 6.1 (Canonical trust (fixpoint form))**.** Given a well-founded premise relation, the **canonical trust** $\tau : C \to T$ is defined by well-founded recursion:

$$\tau(x) = \bigvee_{f \in \mathrm{stepsInto}(x)} \mathrm{stepGrade}(\tau, f),$$

where $\bigvee$ is the `cmax`-fold with initial value $\bot$. The internal definition uses a `dite` guard defaulting to $\bot$ off the premise relation (erased by `tau_eq`).

Lean.

```lean
noncomputable def tau (wf : WellFounded S.Prem) : C → T :=
  wf.fix fun x rec ⇒
    ((S.stepsInto x).map fun f ⇒
      ((S.prems f).map fun y ⇒
        if h : S.Prem y x then rec y h else Chain.bot).foldr
          cmin (S.grade f)).foldr
      cmax Chain.bot
```

Haskell.

```haskell
-- tau s x = foldr cmax bot
--             (map (\f → foldr cmin (grade s f)
--                           (map (tau s) (prems s f)))
--               (stepsInto s x))
-- Non-computable in Lean (Classical); in Haskell we assume
   termination.
tau :: System c t → c → t
tau s x = foldr cmax bot
  [foldr cmin (grade s f) (map (tau s) (prems s f))
  | f <- stepsInto s x]
```

6.2 Theorem: `tau_eq`

Theorem 6.2 (Clean unfolding of τ)**.**

$$\tau(x) = (\mathrm{stepsInto}(x).\mathrm{map}(\mathrm{stepGrade}(\tau, -))).\mathrm{foldr}(\mathrm{cmax}, \bot).$$

This is the intended fixpoint equation; the `dite` guard in the definition collapses to this on well-founded input.

Lean.

```lean
theorem tau_eq (wf : WellFounded S.Prem) (x : C) :
    tau S wf x =
      ((S.stepsInto x).map (stepGrade S (tau S wf))).foldr
        cmax Chain.bot
```

Haskell.

```haskell
-- tau_eq: tau s x = foldr cmax bot (map (stepGrade s (tau s)) (
   stepsInto s x))
```

7 Soundness: the certificate lemma (Lemma 2.2)

7.1 Definition: `Sound`

Definition 7.1 (Sound assignment)**.** An assignment $t : C \to T$ is **sound** if every claim trusted above $\bot$ has a node-local certificate: a checking step whose `stepGrade` under t dominates $t(x)$:

$$\mathrm{Sound}(t) \iff \forall x : C,\ t(x) \neq \bot \ \Rightarrow\ \exists f \in \mathrm{stepsInto}(x),\ t(x) \leq \mathrm{stepGrade}(t, f).$$

This is precisely the deployed boundary rule: the validator checks exactly this predicate at each node.

Lean.

```
def Sound (t : C → T) : Prop :=
  ∀ x : C, t x ≠ Chain.bot →
    exists f, f in S.stepsInto x /\ t x ≤ stepGrade S t f
```

Haskell.

```
sound :: System c t → (c → t) → Bool
sound s t = all check (claims s)
  where
    check x = t x == bot ||
              any (\f → t x <= stepGrade s t f) (stepsInto s x)
```

7.2 Theorem: `sound_le_tau` (Lemma 2.2 ii)

Theorem 7.2 (Every sound assignment is bounded by τ)**.** If t is sound, then $t(x) \leq \tau(x)$ for all x:

$$\mathrm{Sound}(t) \ \Rightarrow\ \forall x,\ t(x) \leq \tau(x).$$

Node-local certificates imply the global bound; claiming above τ is trust inflation and must fail the certificate somewhere.

Lean.

```
theorem sound_le_tau (wf : WellFounded S.Prem) {t : C → T}
    (hs : Sound S t) : ∀ x, t x ≤ tau S wf x
```

Haskell.

```
-- sound_le_tau: sound s t ⇒ forall x, t x <= tau s x
```

7.3 Theorem: `tau_sound` (Lemma 2.2 i)

Theorem 7.3 (τ is sound)**.** The canonical trust is itself sound:

$$\mathrm{Sound}(\tau).$$

This uses `foldr_max_cases` (attainment), hence linearity of the chain. Together with `sound_le_tau`: τ is the *greatest* sound assignment.

Lean.

```
theorem tau_sound (wf : WellFounded S.Prem) :
    Sound S (tau S wf)
```

Haskell.

```
-- tau_sound: sound s (tau s)
```

8 Derivations: max-min semantics agrees with fixpoint

8.1 Inductive type: `Deriv`

Definition 8.1 (Derivation)**.** A **derivation** of claim x is a checking step together with derivations of all its premises (a well-founded tree):

Lean.
```
inductive Deriv (S : System C T) : C → Type where
  | node (f : S.Step) (sub : ∀ y, y in S.prems f → Deriv S y) :
      Deriv S (S.concl f)
```

Haskell.
```
data Deriv c t = Node (Step c t)
                     (Map c (Deriv c t))    -- one sub-derivation per
                        premise
```

8.2 Definition: `foldPrems`

Definition 8.2 (Membership-aware premise fold)**.** A variant of `cmin`-fold over a list of claims, where the value function receives a membership proof. Used to keep `derivGrade` structurally recursive.

Lean.
```
def foldPrems : (l : List C) → (∀ y, y in l → T) → T → T
  | [], _, init ⇒ init
  | y :: t, g, init ⇒
      cmin (g y (.head t))
           (foldPrems t (fun z hz ⇒ g z (.tail y hz)) init)
```

Haskell.
```
foldPrems :: [c] → (c → t) → t → t
foldPrems [] _ init = init
foldPrems (y:ys) g init = cmin (g y) (foldPrems ys g init)
-- (Membership proof is erased in Haskell.)
```

8.3 Definition: `derivGrade`

Definition 8.3 (Derivation grade)**.** The **grade** of a derivation is the meet of every step grade in the tree — the weakest link across all checking steps used:

$$\mathrm{derivGrade}(\texttt{node}\, f\, \mathrm{sub}) = \mathrm{foldPrems}(\mathrm{prems}(f),\ y \mapsto \mathrm{derivGrade}(\mathrm{sub}(y)),\ \mathrm{grade}(f)).$$

Lean.
```
noncomputable def derivGrade : {x : C} → Deriv S x → T :=
  fun d ⇒
    Deriv.rec (motive := fun _ _ ⇒ T)
      (fun f _ ih ⇒
        foldPrems (S.prems f) ih (S.grade f)) d
```

Haskell.
```
derivGrade :: System c t → Deriv c t → t
derivGrade s (Node f sub) =
  foldPrems (prems s f) (derivGrade s . sub) (grade s f)
```

8.4 Theorem: `derivGrade_node`

Theorem 8.4 (Computation rule for derivGrade)**.**

$$\mathrm{derivGrade}(\texttt{node}\, f\, \mathrm{sub}) = \mathrm{foldPrems}(\mathrm{prems}(f),\ y\, h_y \mapsto \mathrm{derivGrade}(\mathrm{sub}(y, h_y)),\ \mathrm{grade}(f)).$$

Lean.

```
theorem derivGrade_node (f : S.Step)
    (sub : ∀ y, y in S.prems f → Deriv S y) :
    derivGrade S (.node f sub) =
      foldPrems (S.prems f)
        (fun y hy ⇒ derivGrade S (sub y hy)) (S.grade f)
```

Haskell.

```
-- derivGrade_node: derivGrade (Node f sub) =
--   foldPrems (prems f) (derivGrade . sub) (grade f)
-- This is just the definition; stated as a lemma for rewriting.
```

8.5 Theorems: `foldPrems_le` and `le_foldPrems`

Theorem 8.5 (foldPrems vs cmin-fold)**.** These bridge lemmas relate `foldPrems` (which carries membership proofs) to the ordinary `cmin`-fold over `List.map`:

If $\forall y\,(h : y \in l),\ g(y,h) \le t(y),$ then $\mathrm{foldPrems}(l,g,\mathrm{init}) \le (l.\mathrm{map}\,t).\mathrm{foldr}(\mathrm{cmin},\mathrm{init})$. (`foldPrems_le`)

If $\forall y\,(h : y \in l),\ t(y) \le g(y,h),$ then $(l.\mathrm{map}\,t).\mathrm{foldr}(\mathrm{cmin},\mathrm{init}) \le \mathrm{foldPrems}(l,g,\mathrm{init})$. (`le_foldPrems`)

Lean.

```
theorem foldPrems_le {t : C → T} {init : T} :
    ∀ (l : List C) (g : ∀ y, y in l → T),
    (∀ y (hy : y in l), g y hy ≤ t y) →
    foldPrems l g init ≤ (l.map t).foldr cmin init

theorem le_foldPrems {t : C → T} {init : T} :
    ∀ (l : List C) (g : ∀ y, y in l → T),
    (∀ y (hy : y in l), t y ≤ g y hy) →
    (l.map t).foldr cmin init ≤ foldPrems l g init
```

Haskell.

```
-- foldPrems_le: g <= t pointwise ⇒ foldPrems l g init <= foldr cmin
   init (map t l)
-- le_foldPrems: t <= g pointwise ⇒ foldr cmin init (map t l) <=
   foldPrems l g init
```

8.6 Theorem: `derivGrade_le_tau`

Theorem 8.6 (Every derivation's grade is below τ)**.** For any $x : C$ and derivation d of x,

$$\mathrm{derivGrade}(d) \le \tau(x).$$

τ dominates the max-min semantics from above.

Lean.

```
theorem derivGrade_le_tau (wf : WellFounded S.Prem) :
    ∀ {x : C} (d : Deriv S x),
    derivGrade S d ≤ tau S wf x
```

Haskell.

```
-- derivGrade_le_tau: derivGrade d <= tau x
-- for any derivation d of x.
```

8.7 Theorem: `tau_le_derivGrade`

Theorem 8.7 (τ is attained by some derivation)**.** If $\tau(x) \neq \bot$, then there exists a derivation d of x with $\tau(x) \le \mathrm{derivGrade}(d)$.

Together with `derivGrade_le_tau`: $\tau(x)$ equals the supremum of derivation grades, so the fixpoint form and the derivation-sup form agree.

Lean.
```
theorem tau_le_derivGrade (wf : WellFounded S.Prem) :
    ∀ x : C, tau S wf x ≠ Chain.bot →
    exists d : Deriv S x, tau S wf x ≤ derivGrade S d
```

Haskell.
```
-- tau_le_derivGrade: tau x /= bot ⇒
--   exists derivation d of x with tau x <= derivGrade d
```

9 Morphisms and transport of trust

9.1 Structure: `Mor`

Definition 9.1 (Morphism of trust systems)**.** A **morphism** $M : S \to S'$ between trust systems (over possibly different claim types C, C' and chain types T, T') consists of:

- onClaim : $C \to C'$,
- onStep : $S.\texttt{Step} \to S'.\texttt{Step}$,
- $\phi : T \to T'$ (monotone, strict at bottom: $\phi(\bot) = \bot$),
- premise compatibility: $S'.\text{prems}(\text{onStep}(f)) = \text{prems}(f).\text{map}(\text{onClaim})$,
- grade laxity: $\phi(S.\text{grade}(f)) \le S'.\text{grade}(\text{onStep}(f))$,
- step membership: $f \in \text{stepsInto}(x) \Rightarrow \text{onStep}(f) \in \text{stepsInto}'(\text{onClaim}(x))$.

The strictness $\phi(\bot) = \bot$ is a correction to the prose Definition 3.1, discovered by this formalisation: without it, transport fails on underivable claims.

Lean.
```
structure Mor (S : System C T) (S' : System C' T') where
  onClaim  : C → C'
  onStep   : S.Step → S'.Step
  phi      : T → T'
  phi_mono : ∀ {a b : T}, a ≤ b → phi a ≤ phi b
  phi_bot  : phi Chain.bot = Chain.bot
  prems_eq : ∀ f, S'.prems (onStep f) = (S.prems f).map onClaim
  grade_le : ∀ f, phi (S.grade f) ≤ S'.grade (onStep f)
  steps_mem : ∀ {x : C} {f : S.Step},
    f in S.stepsInto x → onStep f in S'.stepsInto (onClaim x)
```

Haskell.
```
data Mor c t c' t' = Mor
  { onClaim  :: c → c'
  , onStep   :: Step c t → Step c' t'
  , phi      :: t → t'
  -- phi_mono, phi_bot, prems_eq, grade_le, steps_mem are laws
  }
```

9.2 Theorem: `tau_transport` (Lemma 3.2)

Theorem 9.2 (Trust transports laxly along morphisms)**.** For any morphism $M : S \to S'$ and well-founded premise relations,

$$\forall x : C, \quad M.\phi(\tau_S(x)) \le \tau_{S'}(M.\text{onClaim}(x)).$$

Importing along a morphism never claims more than ϕ of the source's canonical trust.

Lean.
```
theorem tau_transport {S : System C T} {S' : System C' T'}
    (M : Mor S S')
    (wf : WellFounded S.Prem) (wf' : WellFounded S'.Prem) :
    ∀ x : C,
    M.phi (tau S wf x) ≤ tau S' wf' (M.onClaim x)
```

Haskell.
```
-- tau_transport: phi m (tau s x) <= tau s' (onClaim m x)
-- for any morphism m :: Mor s s'.
```

10 The circular-citation counterexample (Remark 2.3)

10.1 Inductive type: `Two`

Definition 10.1 (Two-element chain)**.** The two-level chain $\{\texttt{lo}, \texttt{hi}\}$ with $\texttt{lo} < \texttt{hi}$, $\bot = \texttt{lo}$.

Lean.
```
inductive Two where
  | lo
  | hi
deriving DecidableEq

instance : Chain Two where
  le a b := a = .lo \/ b = .hi
  bot := .lo
  -- le_refl, le_trans, le_antisymm, le_total, bot_le, dec all proved
```

Haskell.
```
data Two = Lo | Hi deriving (Eq, Ord, Show, Bounded)
-- Lo < Hi; bot = Lo.
```

10.2 Definition: `cyc`

Definition 10.2 (Circular citation system)**.** A trust system on `Bool` claims: the only step into claim x has the single premise $\neg x$, graded `hi`. This encodes a two-claim cycle: each claim is justified by the other.

Lean.
```
def cyc : System Bool Two where
  Step       := Bool
  concl      := id
  prems f    := [!f]
  grade _    := .hi
  stepsInto x := [x]
  mem_stepsInto := ...
  stepsInto_mem f := .head []
```

Haskell.
```
cyc :: System Bool Two
cyc = System
  { concl = id, prems = \f → [not f]
  , grade = const Hi, stepsInto = \x → [x]
  , ... }
```

10.3 Theorem: `cyc_sound_top`

Theorem 10.3 (Constant-`hi` is sound on `cyc`)**.** The assignment $x \mapsto$ `hi` is sound: every node-local certificate passes.

Lean.
```
theorem cyc_sound_top : Sound cyc (fun _ ⇒ Tw ∘ hi)
```

Haskell.
```
-- sound cyc (const Hi) = True
-- (Each step is its own certificate: grade hi, premise is the other
    claim
--  also assigned hi.)
```

10.4 Theorem: `cyc_no_deriv`

Theorem 10.4 (No claim in `cyc` has any derivation)**.**

$$\forall x : \texttt{Bool},\ \texttt{Deriv}\,\texttt{cyc}\,x \to \texttt{False}.$$

The max-min semantics is vacuous: there are no finite derivation trees.

Lean.
```
theorem cyc_no_deriv : ∀ x : Bool, Deriv cyc x → False
```

Haskell.
```
-- Any attempt to build a Deriv for cyc loops forever.
```

10.5 Theorem: `cyc_not_wf`

Theorem 10.5 (`cyc`'s premise relation is not well-founded)**.** $\neg$`WellFounded(cyc.Prem)`.

Therefore the certificate lemma does not apply to `cyc` — as it must not, since `cyc_sound_top` shows that `hi` passes every local check while `cyc_no_deriv` shows no derivation exists.

Lean.
```
theorem cyc_not_wf : not WellFounded cyc.Prem
```

Haskell.
```
-- cyc's dependency graph has a cycle (true → false → true → ...),
-- so the premise relation is not well-founded.
```

11 Consolidations

11.1 Structure: `Consolidation`

Definition 11.1 (Consolidation)**.** A **consolidation** between trust systems S (over C) and S' (over C') over the *same* check-chain T consists of:

Shapes forward:

- onClaim : $C \to C'$,
- onStep : S.`Step` $\to$ S'.`Step`,
- premise compatibility and grade laxity $S.\mathrm{grade}(f) \le S'.\mathrm{grade}(\mathrm{onStep}(f))$,
- step membership.

Positions backward: for each $x : C$, every target step $f' \in S'.\text{stepsInto}(\text{onClaim}(x))$ pulls back to a source step $\text{posn}(f', h) \in S.\text{stepsInto}(x)$, with premise correspondence and *citation condition*: $S'.\text{grade}(f') \leq S.\text{grade}(\text{posn}(f', h))$.

The citation condition captures the notion of a cached grade: the consolidated system records a grade at most as strong as the source's. Note `posn` is indexed by the preimage x: when consolidation merges claims, every justification must pull back on each side.

Lean.

```
structure Consolidation (S : System C T) (S' : System C' T) where
  -- shapes forward
  onClaim    : C → C'
  onStep     : S.Step → S'.Step
  prems_eq   : ∀ f, S'.prems (onStep f) = (S.prems f).map onClaim
  grade_le   : ∀ f, S.grade f ≤ S'.grade (onStep f)
  steps_mem  : ∀ {x : C} {f : S.Step},
    f in S.stepsInto x → onStep f in S'.stepsInto (onClaim x)
  -- positions backward
  posn       : ∀ {x : C} (f' : S'.Step),
    f' in S'.stepsInto (onClaim x) → S.Step
  posn_mem   : ∀ {x : C} {f' : S'.Step}
    (h : f' in S'.stepsInto (onClaim x)),
    posn f' h in S.stepsInto x
  posn_prems : ∀ {x : C} {f' : S'.Step}
    (h : f' in S'.stepsInto (onClaim x)),
    S'.prems f' = (S.prems (posn f' h)).map onClaim
  posn_grade : ∀ {x : C} {f' : S'.Step}
    (h : f' in S'.stepsInto (onClaim x)),
    S'.grade f' ≤ S.grade (posn f' h)
```

Haskell.

```
data Consolidation c t c' = Consolidation
  { onClaim   :: c → c'
  , onStep    :: Step c t → Step c' t
  , posn      :: c → Step c' t → Step c t
  -- prems_eq, grade_le, steps_mem, posn_mem, posn_prems, posn_grade
     are laws
  }
```

Fidelity note. The `posn` field takes a membership proof $h : f' \in \text{stepsInto}'(\text{onClaim}(x))$ as an argument. This is the dependent type that records *which* preimage x we are pulling back to. When consolidation merges multiple claims, `posn` must be defined separately for each preimage; the Haskell approximation adds x as an extra argument to `posn`, but loses the membership-proof dependency.

11.2 Definition: `Consolidation.toMor`

Definition 11.2 (Forward half of consolidation is a morphism)**.** The forward data of a consolidation $K : S \to S'$ at $\phi = \text{id}$ assembles into a morphism `Mor S S'`.

Lean.

```
def Consolidation.toMor {S : System C T} {S' : System C' T}
    (K : Consolidation S S') : Mor S S' where
  onClaim   := K.onClaim
  onStep    := K.onStep
  phi       := id
  phi_mono  := fun h ⇒ h
  phi_bot   := rfl
```

```
  prems_eq  := K.prems_eq
  grade_le  := K.grade_le
  steps_mem := K.steps_mem
```

Haskell.

```
toMor :: Consolidation c t c' → Mor c t c' t
toMor k = Mor (onClaim k) (onStep k) id
```

11.3 Theorem: `Consolidation.tau_le`

Theorem 11.3 (Position map forbids trust inflation)**.** A consolidation $K : S \to S'$ satisfies

$$\forall x : C, \quad \tau_{S'}(\mathrm{onClaim}(x)) \le \tau_S(x).$$

The position map pulls every target justification back to a source justification, preventing the consolidated system from claiming more than the source.

Lean.

```
theorem Consolidation.tau_le {S : System C T} {S' : System C' T}
    (K : Consolidation S S')
    (wf : WellFounded S.Prem) (wf' : WellFounded S'.Prem) :
    ∀ x : C, tau S' wf' (K.onClaim x) ≤ tau S wf x
```

Haskell.

```
-- tau_le: tau s' (onClaim k x) <= tau s x
-- for all x, given consolidation k :: Consolidation s s'.
```

11.4 Theorem: `tau_consolidate` (Theorem 3)

Theorem 11.4 (Consolidation preservation (Theorem 3))**.** A consolidation $K : S \to S'$ with a genuine position map preserves canonical trust on the nose:

$$\forall x : C, \quad \tau_{S'}(\mathrm{onClaim}(x)) = \tau_S(x).$$

Forward inequality: shapes transport trust laxly (`tau_transport` at $\phi = \mathrm{id}$). Backward inequality: positions pull every target justification back (`tau_le`).

Lean.

```
theorem tau_consolidate {S : System C T} {S' : System C' T}
    (K : Consolidation S S')
    (wf : WellFounded S.Prem) (wf' : WellFounded S'.Prem) :
    ∀ x : C,
    tau S' wf' (K.onClaim x) = tau S wf x
```

Haskell.

```
-- tau_consolidate: tau s' (onClaim k x) == tau s x
-- for all x, given a full consolidation k (with position map).
```

12 Composition of consolidations

12.1 Definition: `Consolidation.id`

Definition 12.1 (Identity consolidation)**.** The identity consolidation on S: shapes and positions both identity, grades exact ($S.\mathrm{grade}(f) \le S.\mathrm{grade}(f)$ by reflexivity).

Lean.

```
protected def Consolidation.id (S : System C T) :
    Consolidation S S where
  onClaim := id
  onStep  := id
  prems_eq f := (List.map_id (S.prems f)).symm
  grade_le f := Chain.le_refl (S.grade f)
  steps_mem h := h
  posn f' _ := f'
  posn_mem h := h
  posn_prems {_ f'} _ := (List.map_id (S.prems f')).symm
  posn_grade {_ f'} _ := Chain.le_refl (S.grade f')
```

Haskell.

```
consolidationId :: Consolidation c t c
consolidationId = Consolidation id id (\_ f' → f')
```

12.2 Definition: `Consolidation.comp`

Definition 12.2 (Composition of consolidations)**.** Given $K : S \to S'$ and $L : S' \to S''$, their composite $K \cdot L : S \to S''$ has:

- shapes: onClaim $= L.\text{onClaim} \circ K.\text{onClaim}$, onStep $= L.\text{onStep} \circ K.\text{onStep}$;
- positions: $\text{posn}(f'', h) = K.\text{posn}(L.\text{posn}(f'', h), L.\text{posn_mem}(h))$ (contravariant — pull back through L first, then through K);
- grades: by transitivity of $\leq$.

Lean.

```
protected def Consolidation.comp
    {S : System C T} {S' : System C' T} {S'' : System C'' T}
    (K : Consolidation S S') (L : Consolidation S' S'') :
    Consolidation S S'' where
  onClaim x := L.onClaim (K.onClaim x)
  onStep f  := L.onStep (K.onStep f)
  prems_eq f := by rw [L.prems_eq, K.prems_eq, List.map_map]; rfl
  grade_le f := Chain.le_trans (K.grade_le f) (L.grade_le (K.onStep f)
     )
  steps_mem h := L.steps_mem (K.steps_mem h)
  posn f'' h  := K.posn (L.posn f'' h) (L.posn_mem h)
  posn_mem h  := K.posn_mem (L.posn_mem h)
  posn_prems h := by
    rw [L.posn_prems h, K.posn_prems (L.posn_mem h), List.map_map];
       rfl
  posn_grade h := Chain.le_trans (L.posn_grade h)
                    (K.posn_grade (L.posn_mem h))
```

Haskell.

```
consolidationComp :: Consolidation c t c'
                  → Consolidation c' t c''
                  → Consolidation c t c''
consolidationComp k l = Consolidation
  { onClaim = onClaim l . onClaim k
  , onStep  = onStep l . onStep k
  , posn    = \x f'' → posn k (onClaim k x) (posn l (onClaim l (
     onClaim k x)) f'')
  }
```

12.3 Theorem: `tau_consolidate_comp`

Theorem 12.3 (End-to-end preservation through a pipeline)**.** For composable consolidations $K : S \to S'$ and $L : S' \to S''$:

$$\forall x : C, \quad \tau_{S''}((K \cdot L).\text{onClaim}(x)) = \tau_S(x).$$

This factors as $\tau_{S''}(L.\text{onClaim}(K.\text{onClaim}(x))) = \tau_{S'}(K.\text{onClaim}(x)) = \tau_S(x)$, so end-to-end trust preservation is the conjunction of the per-layer equations.

Lean.

```
theorem tau_consolidate_comp
    {S : System C T} {S' : System C' T} {S'' : System C'' T}
    (K : Consolidation S S') (L : Consolidation S' S'')
    (wf : WellFounded S.Prem)
    (wf' : WellFounded S'.Prem)
    (wf'' : WellFounded S''.Prem) :
    ∀ x : C,
    tau S'' wf'' ((K.comp L).onClaim x) = tau S wf x
```

Haskell.

```
-- tau_consolidate_comp:
--   tau s'' (onClaim (comp k l) x) == tau s x
-- for all x, composable consolidations k, l.
```

13 The shape-only counterexample

13.1 Definitions: `ground` and `layer4`

Definition 13.1 (Shape-only counterexample)**.** Two systems over `Unit` claims, with the two-element chain `Two`:

`ground`: one claim, one `hi`-graded ground check (no premises).

$$\texttt{Step} = \texttt{Unit},\ \text{prems} = [],\ \text{grade} = \texttt{hi},\ \text{stepsInto}(\langle\rangle) = [\langle\rangle].$$

`layer4`: same claim, no steps at all (the shape-only consolidated version: the claim survives but the check is dropped).

$$\texttt{Step} = \texttt{Empty},\ \text{stepsInto}(\langle\rangle) = [].$$

Lean.

```
def ground : System Unit Two where
  Step := Unit
  concl _ := ()
  prems _ := []
  grade _ := .hi
  stepsInto _ := [()]
  mem_stepsInto _ := rfl
  stepsInto_mem _ := .head []

def layer4 : System Unit Two where
  Step := Empty
  concl := Empty.elim
  prems := Empty.elim
  grade := Empty.elim
  stepsInto _ := []
  mem_stepsInto := fun h ⇒ nomatch h
  stepsInto_mem := fun f ⇒ f.elim
```

Haskell.

```haskell
-- ground: one ground check graded Hi; layer4: no steps.
ground :: System () Two
ground = System { step=[()] , concl=const ()
                , prems=const [], grade=const Hi
                , stepsInto=const [()] }

data Void  -- empty type
layer4 :: System () Two
layer4 = System { step=[], concl=absurd
                , prems=absurd, grade=absurd
                , stepsInto=const [] }
```

13.2 Theorems: `ground_wf` and `layer4_wf`

Theorem 13.2 (Well-foundedness of both systems)**.** Both `ground` and `layer4` have well-founded premise relations (both empty).

Lean.

```
theorem ground_wf : WellFounded ground.Prem
theorem layer4_wf : WellFounded layer4.Prem
```

Haskell.

```haskell
-- Both systems have empty premise relations; trivially well-founded.
```

13.3 Theorems: `ground_tau_hi` and `layer4_tau_bot`

Theorem 13.3 (Trust degrades from `hi` to `bot` after shape-only consolidation)**.**

$$\tau_{\texttt{ground}}(\langle\rangle) = \texttt{hi}, \qquad \tau_{\texttt{layer4}}(\langle\rangle) = \texttt{lo} = \bot.$$

This is the formal shadow of the `synthesis-layer4.md` failure: after four shape-only consolidation layers, machine-resolvable citations are gone, and canonical trust collapses to $\bot$.

Lean.

```
theorem ground_tau_hi :
    tau ground ground_wf () = Tw ∘ hi

theorem layer4_tau_bot :
    tau layer4 layer4_wf () = Tw ∘ lo
```

Haskell.

```haskell
-- tau ground () == Hi    (ground check graded Hi, no premises)
-- tau layer4 () == Lo    (no steps → foldr cmax bot [] = bot)
```

14 Translation conventions

1. **Dependent types throughout.** `System.Step` varies between instances; `Consolidation.posn` depends on a membership proof; `Deriv S x` is indexed by the claim x. All of these are approximated by erasing the dependency in Haskell.
2. **Noncomputable definitions.** `tau` and `derivGrade` are marked `noncomputable` because they use `Classical.choice` (via `WellFounded.fix` and `Deriv.rec`). In Haskell they are written as recursive functions; termination is assumed.
3. **The three roles of linearity.** The `Chain` hypothesis (totality) is used at exactly three points: (a) `cmax_cases` (attainment in `tau_sound`), (b) `foldr_max_cases` (attainment

in `tau_le_derivGrade`), and (c) antisymmetry in `tau_le_derivGrade`'s uniqueness argument. All three fail for general lattices.

4. **Phi-bot as a correction.** The `phi_bot` field of `Mor` ($\phi(\bot) = \bot$) is not in the original prose definition. Without it, `tau_transport` fails for claims with no derivation (where $\tau = \bot$) if $\phi(\bot) > \bot$ in the target chain. This is a genuine mathematical correction discovered by Lean.

5. **Consolidation vs morphism.** A `Consolidation` is a `Mor` at $\phi = \mathrm{id}$ plus a backward position map. The extra structure (positions backward) is what yields preservation ($=$) rather than lax transport ($\leq$).

6. **Proof bodies omitted.** Lean checks all proofs.